



SourceTree 써보기 & 내 컴퓨터에 clone 해보기

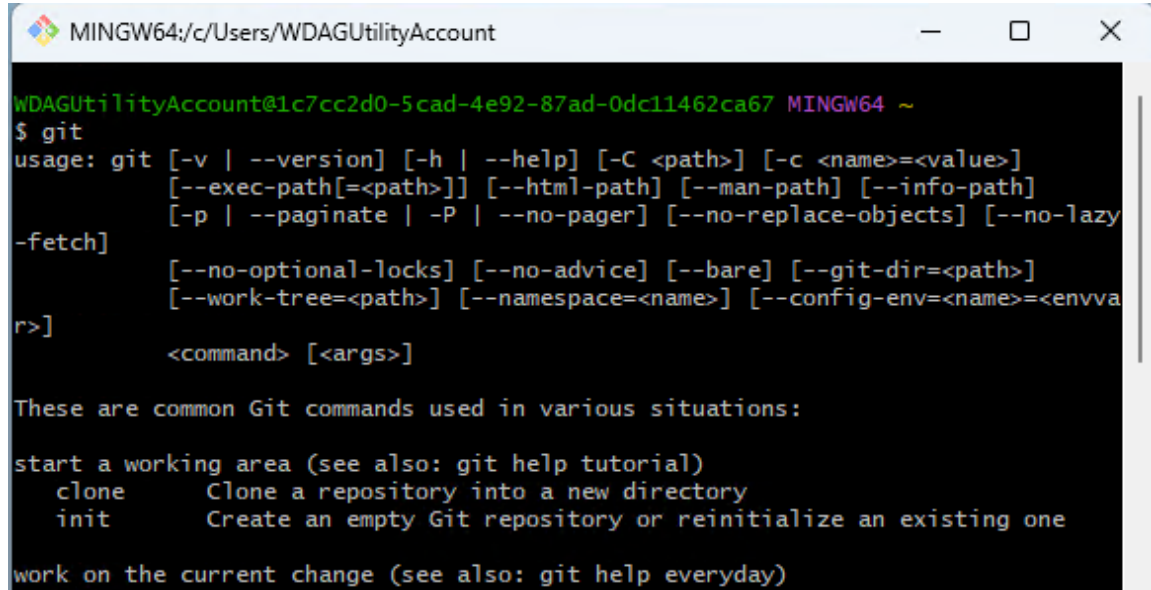
버전 관리의 시작점

- 사용이 어려움
- 그냥 폴더로 관리 하면 안되나?
- 가능함.
- 그런데, 컴퓨터가 해킹 당한다면?!
- 혹은, 컴퓨터가 바뀐다면? (집컴, 회사컴)
- 프로젝트를 쉽게 공유하고 싶다면?
- 여러곳에서 사용가능한 Github이 좋음
- 그러나, 사용방법을 배워야 한다!



자, 이제 시작이야

- Git 레포지토리 관리는 사실 어려움, 왜냐하면...



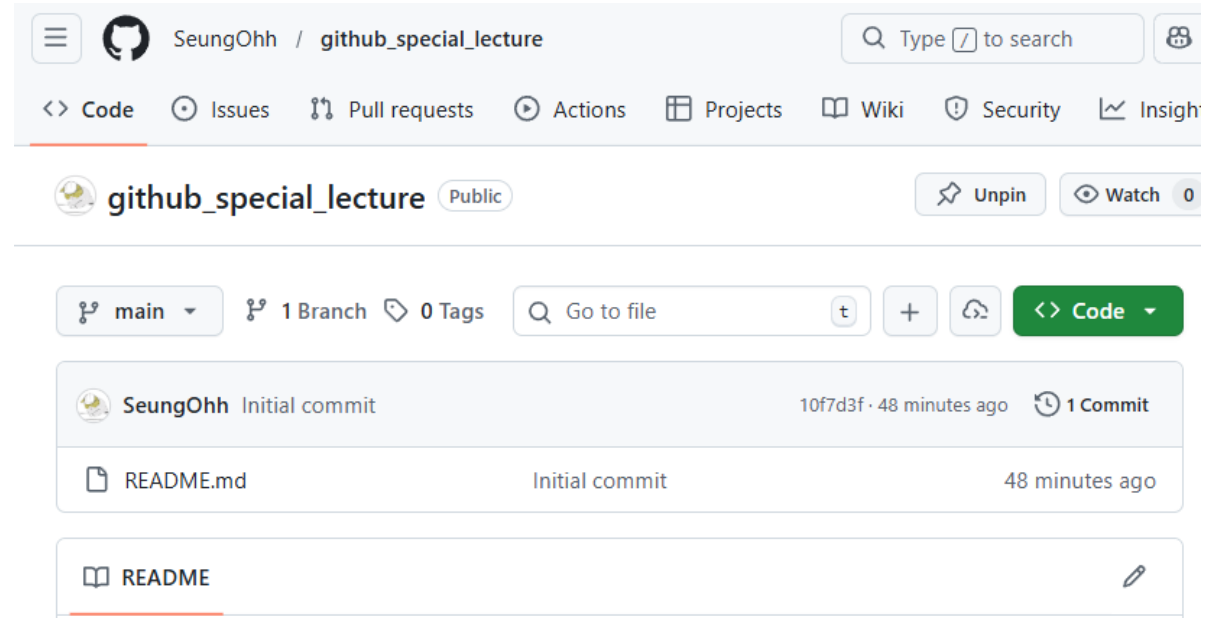
```
MINGW64:/c:/Users/WDAGUtilityAccount
WDAGUtilityAccount@1c7cc2d0-5cad-4e92-87ad-0dc11462ca67 MINGW64 ~
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy
          fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envva
          r>]
          <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
```

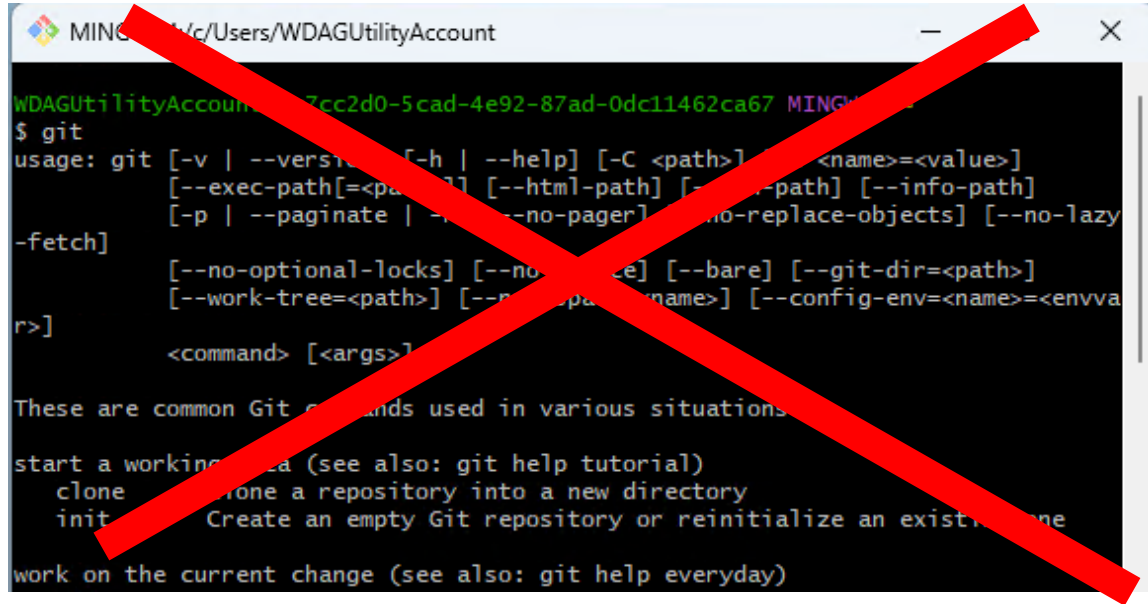
원래, git을 사용하려면 여기서 했어야 함.



웹페이지에서 작업하는건 기능이 부족함.
(단순 수정밖에 안됨)

SourceTree, 깃깃프로그램

- Git 레포지토리 관리는 사실 어려움, 왜냐하면...



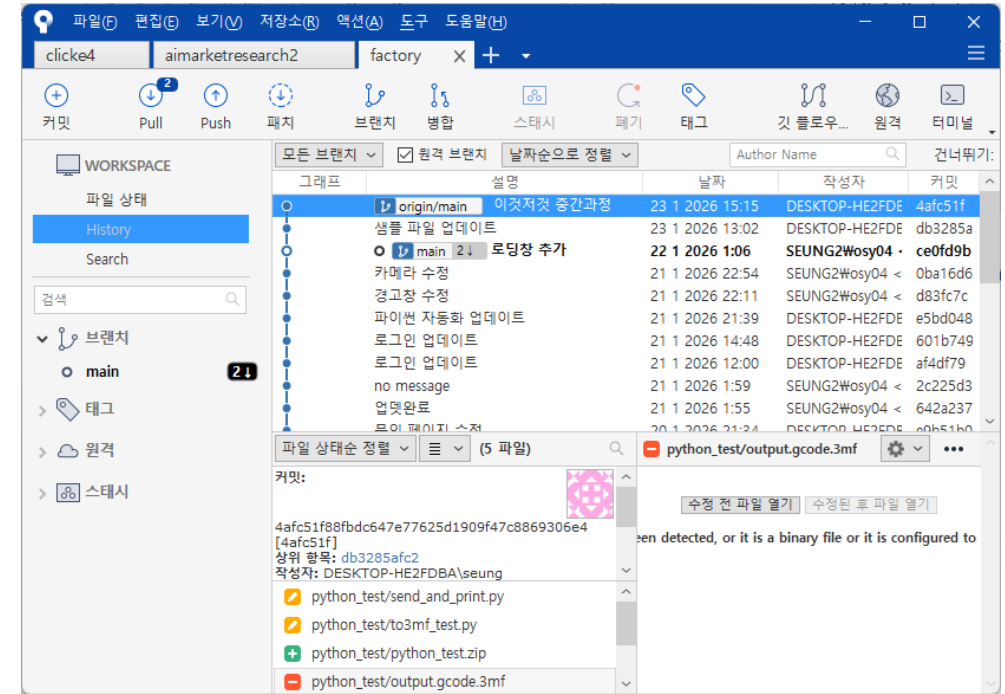
```
WDAGUtilityAccount [C:/Users/WDAGUtilityAccount]
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-C <name>=<value>]
[--exec-path=<path>] [--html-path] [--info-path] [--no-pager]
[--no-optional-checkouts] [--no-replace-objects] [--no-lazy-fetch]
[--no-optional-locks] [--no-optional-checkouts] [--bare] [--git-dir=<path>]
[--work-tree=<path>] [--no-prepare-commit-msg] [--config-env=<name>=<envvar>]
[<command>] [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
clone Clone a repository into a new directory
init Create an empty Git repository or reinitialize an existing one
work on the current change (see also: git help everyday)
```

원래, git을 사용하려면 여기서 했어야 함.

기능의 100%를 사용할 수 있지만, 어려움



sourcetree라고 하는 엄청 좋은 툴을 사용하면 편함!

하지만 기능의 90% 만 사용 가능

SourceTree, 깃프로그램

- Sourcetree를 쓰면 편함

나쁜 답변

```
MINGW64/c/Users/WDAGUtilityAccount

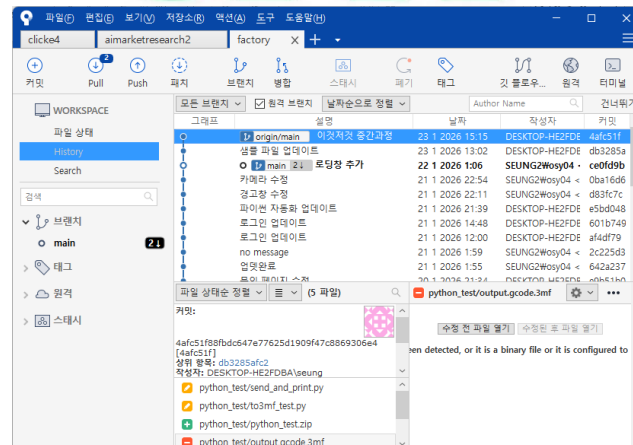
WDAGUtilityAccount@1c7cc2d0-5cad-4e92-87ad-0dc11462ca67 MINGW64 ~
$ git
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
[--exec-path=<path>] [--html-path] [--man-path] [--info-path]
[-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy
-fetch]
[--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
[--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envva
r>]
<command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one
work on the current change (see also: git help everyday)
```

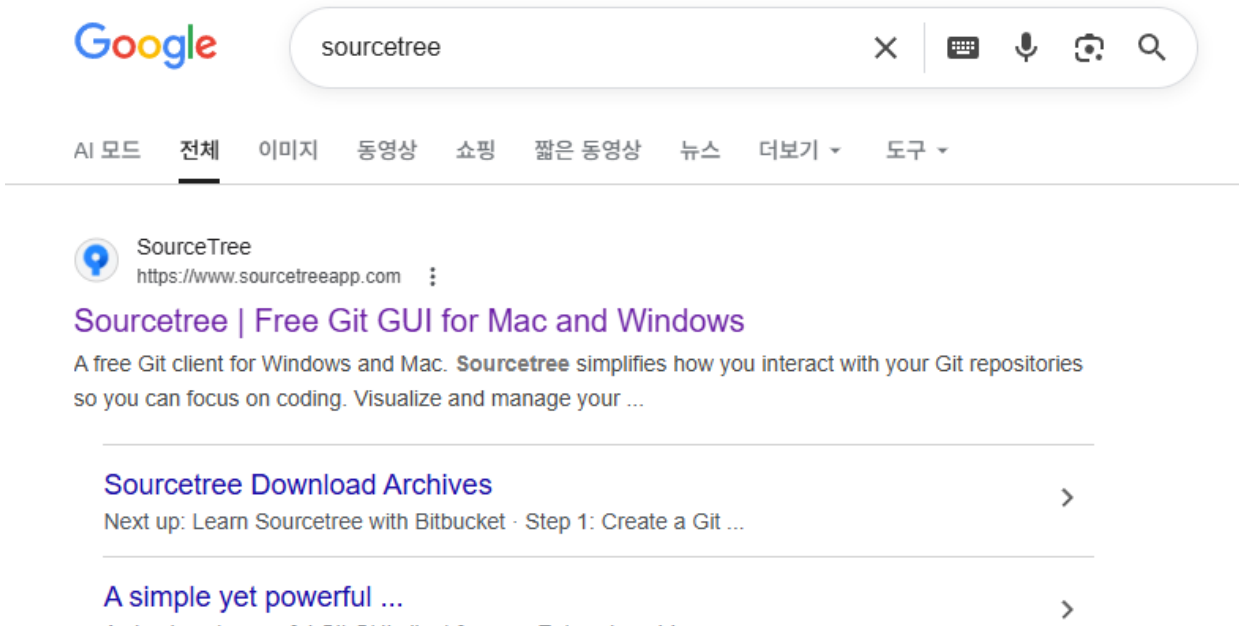
- 시간이 오래 걸림
- 지적허영심을 뽐내는 것 같음
- 지루하고 현학적임
- 논쟁 발생 가능성 100%
- 논쟁에서 밀리면 내 세상이 무너짐

좋은 답변

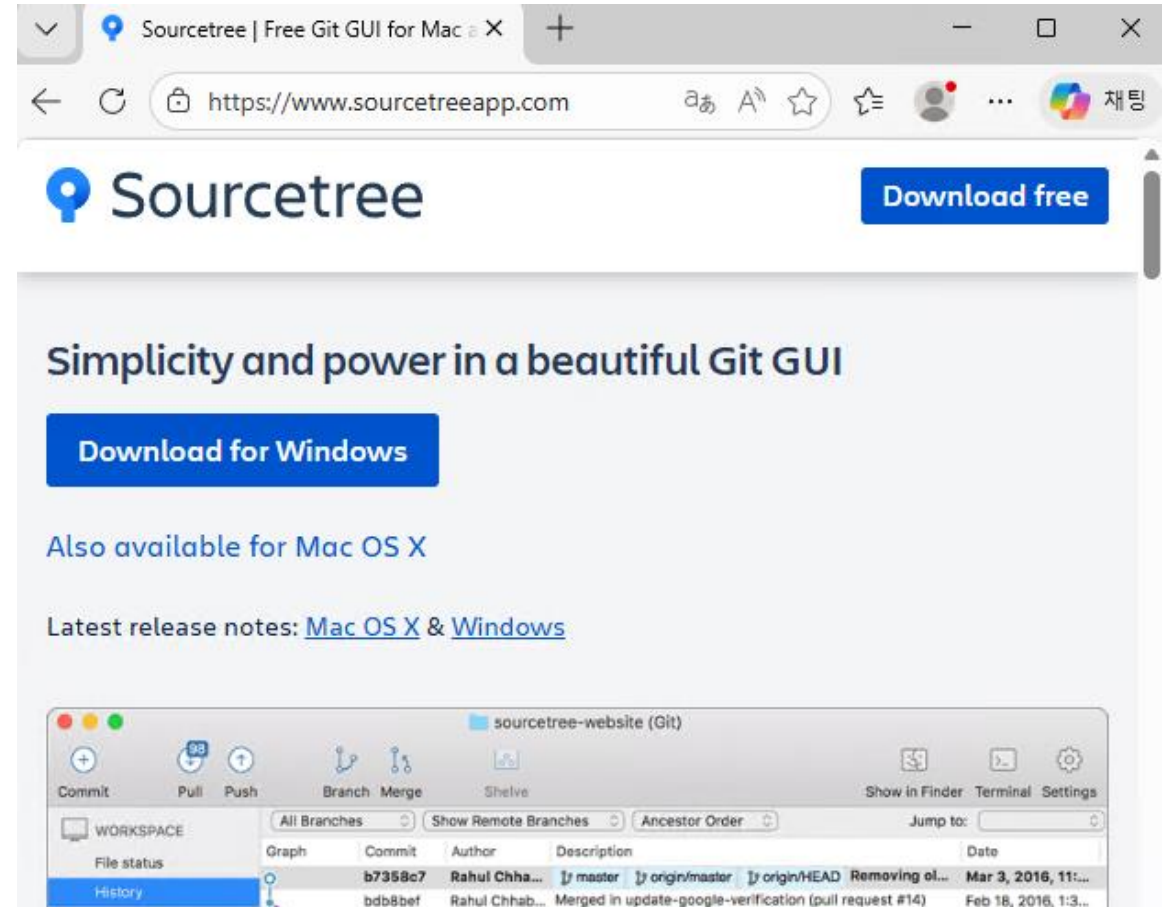


- 금방 끝남
- 오래 기억에 남음
- 팩트임
- 간단하고 사전 지식이 필요X
- 논쟁에서 밀릴 확률 0에 수렴함

SourceTree, 깃프로그램

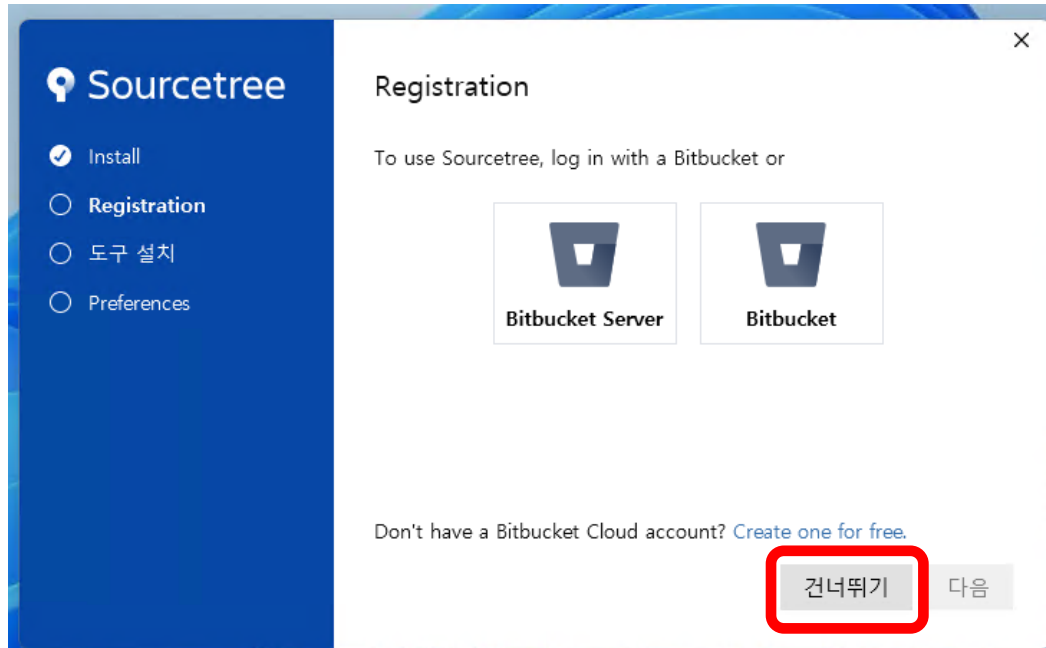


sourcetree 검색하고

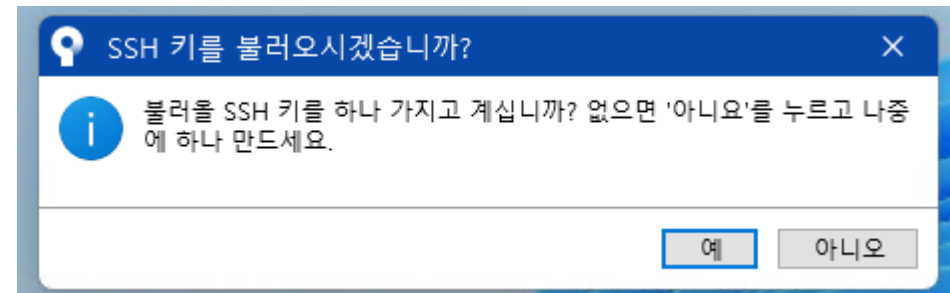


Download for Windows 클릭

SourceTree, 깃갓프로그램



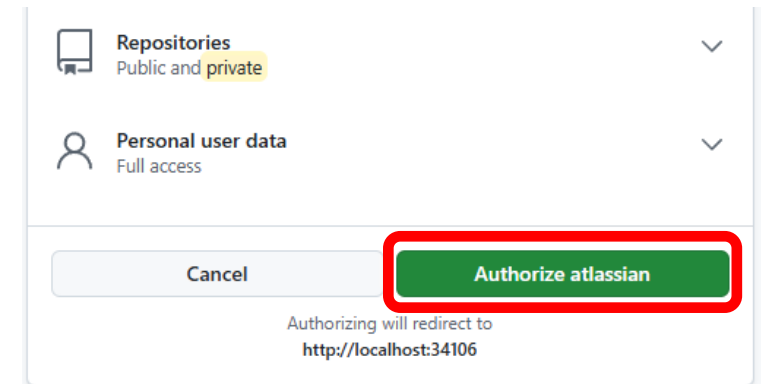
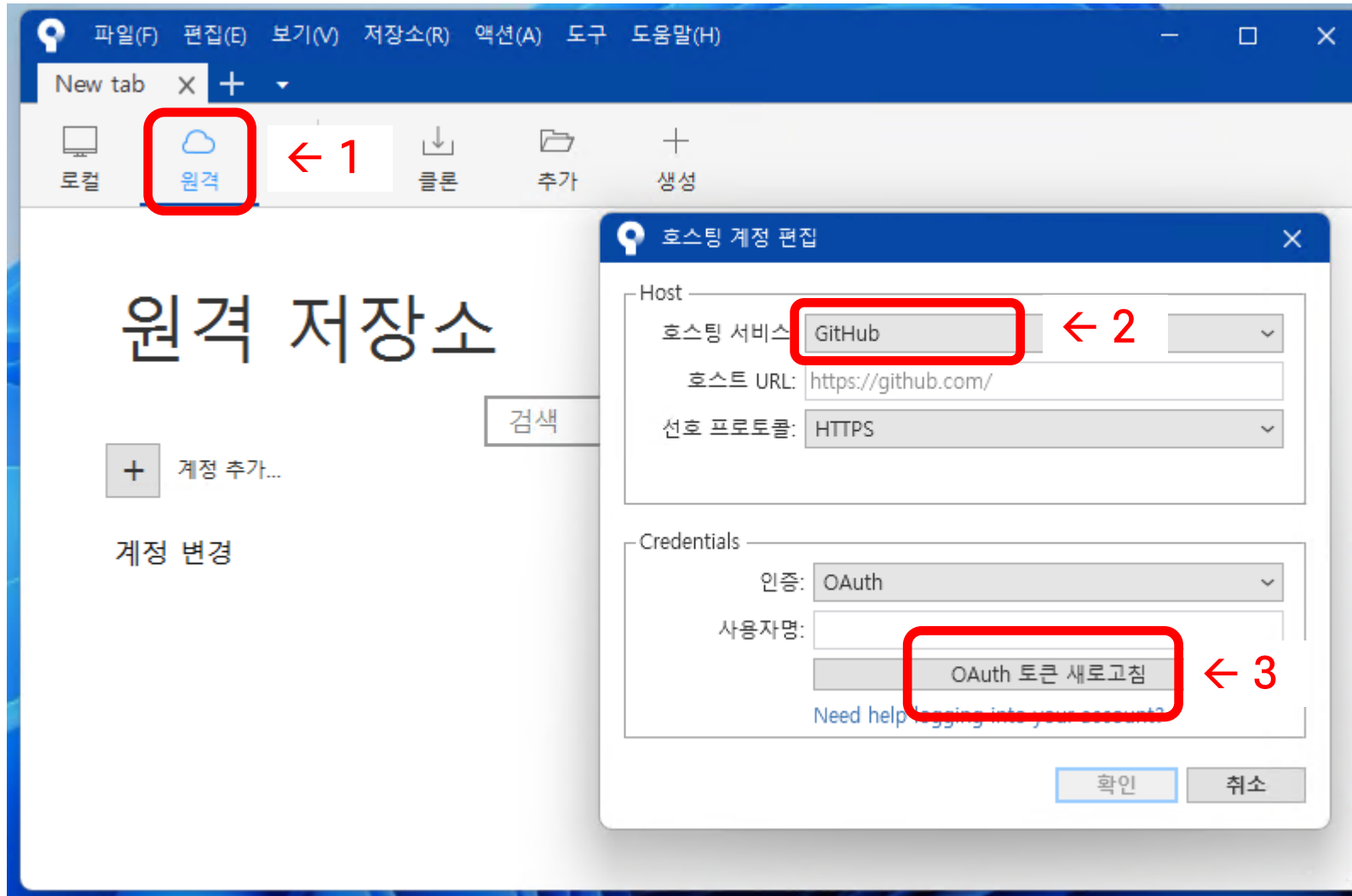
건너뛰기



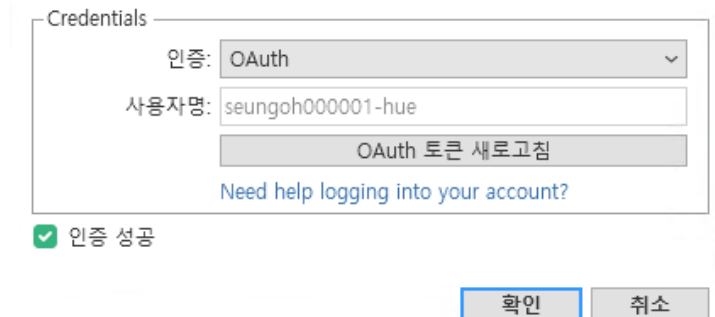
아니오 클릭

SourceTree, 깃갓프로그램

- Sourcetree에서 Github 로그인



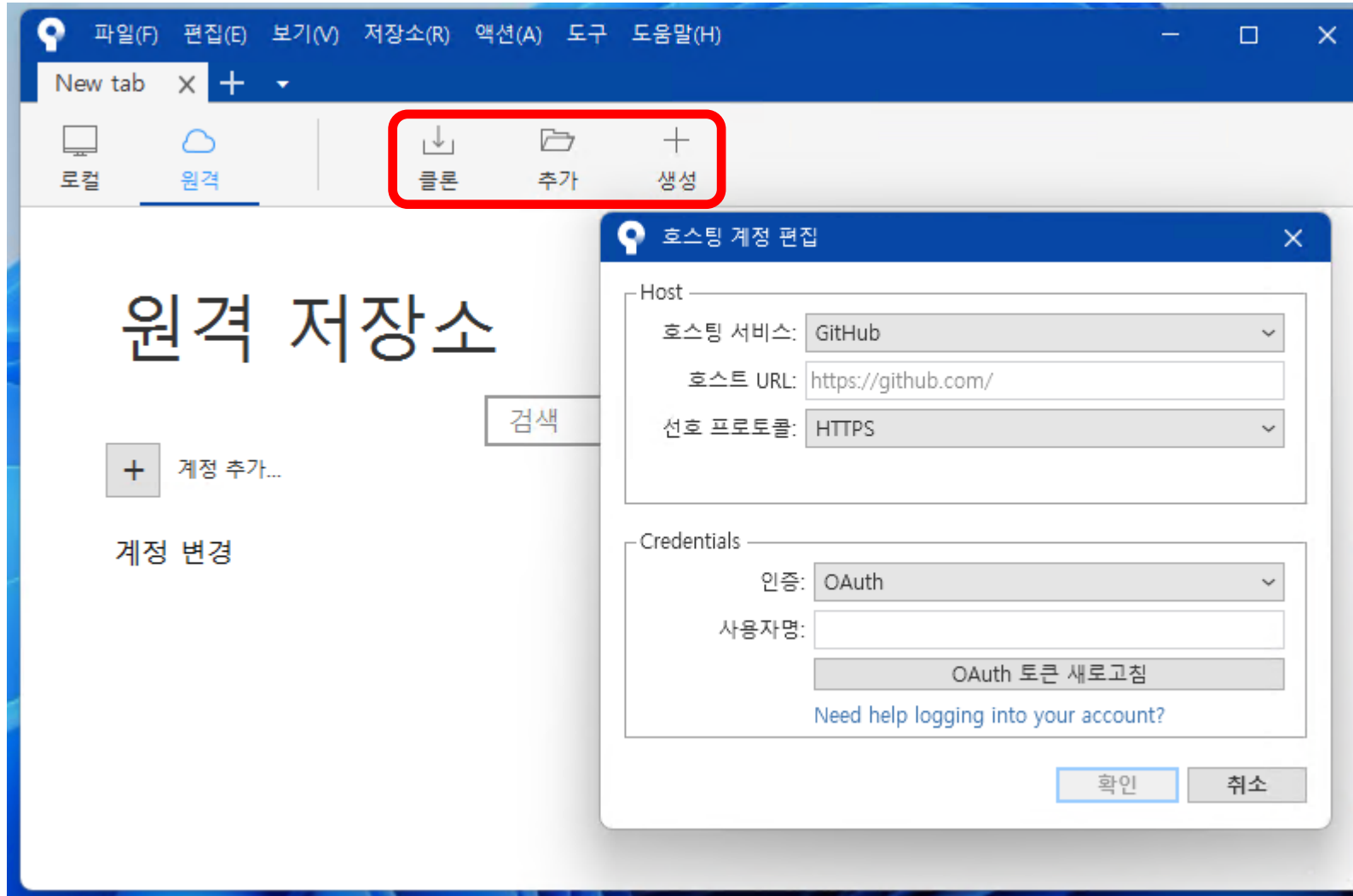
여기를 누르면 완료



로그인 완료!
확인 클릭!

SourceTree, 깃갯프로그램

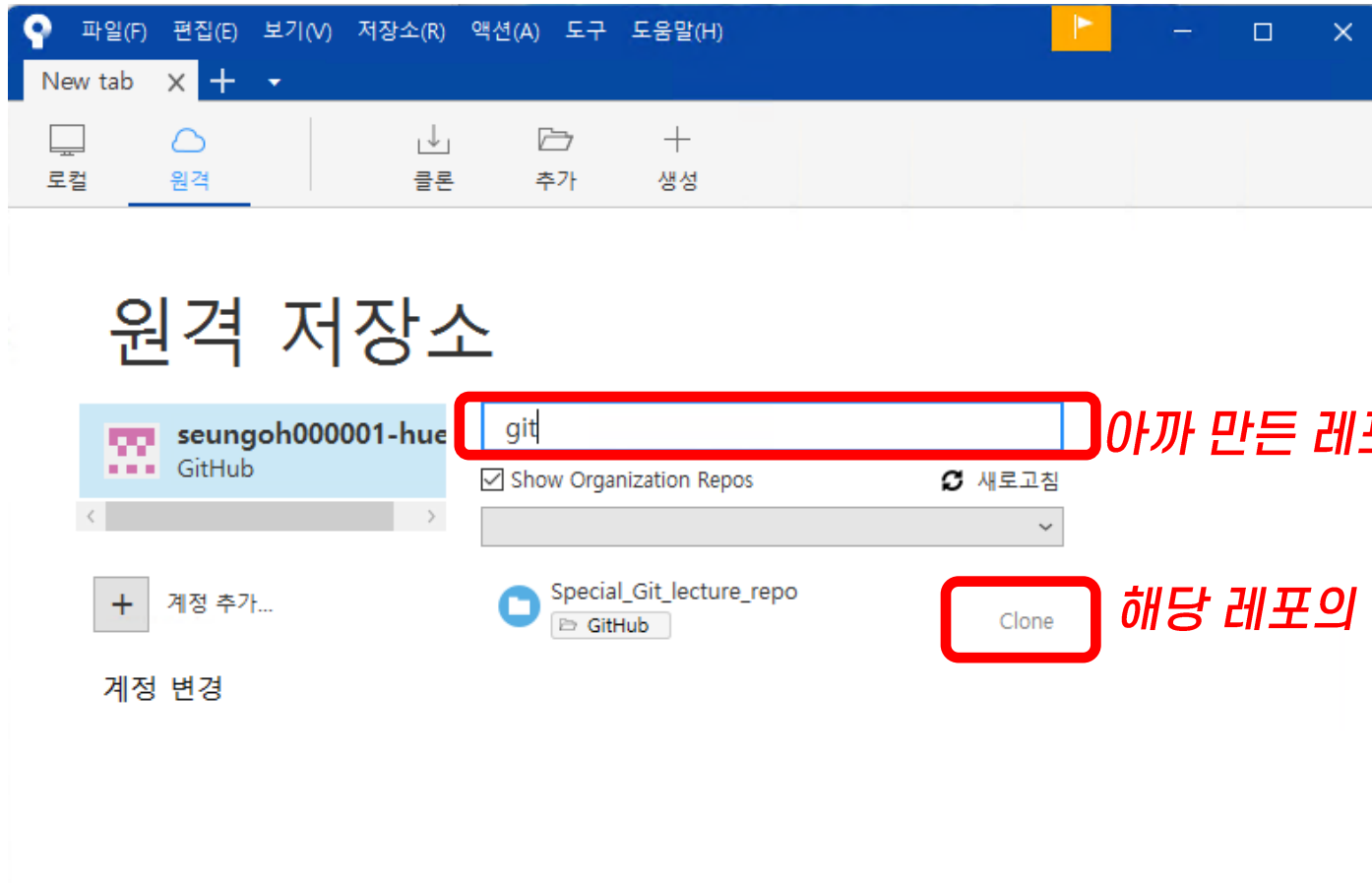
- 새 탭의 클론 / 추가 / 생성, 뭐가 다르지?



- 클론 (Clone) ← 우리는 이걸 씀!
 - GitHub에 이미 있는 저장소를 내 컴퓨터로 복사해서 가져오기
- 추가 (Add)
 - 내 컴퓨터에 이미 있는 git 폴더를 목록에 등록만 함 (복사 X)
- 생성 (Create)
 - 내 컴퓨터의 빈 폴더를 새 git 저장소로 만듦 (GitHub 연결은 나중에)
- 한 줄 요약
 - 클론 = 가져오기 / 추가 = 등록하기 / 생성 = 새로 만들기

SourceTree, 깃갓프로그램

- 아까 만든 Repo를 불러올 수 있음!

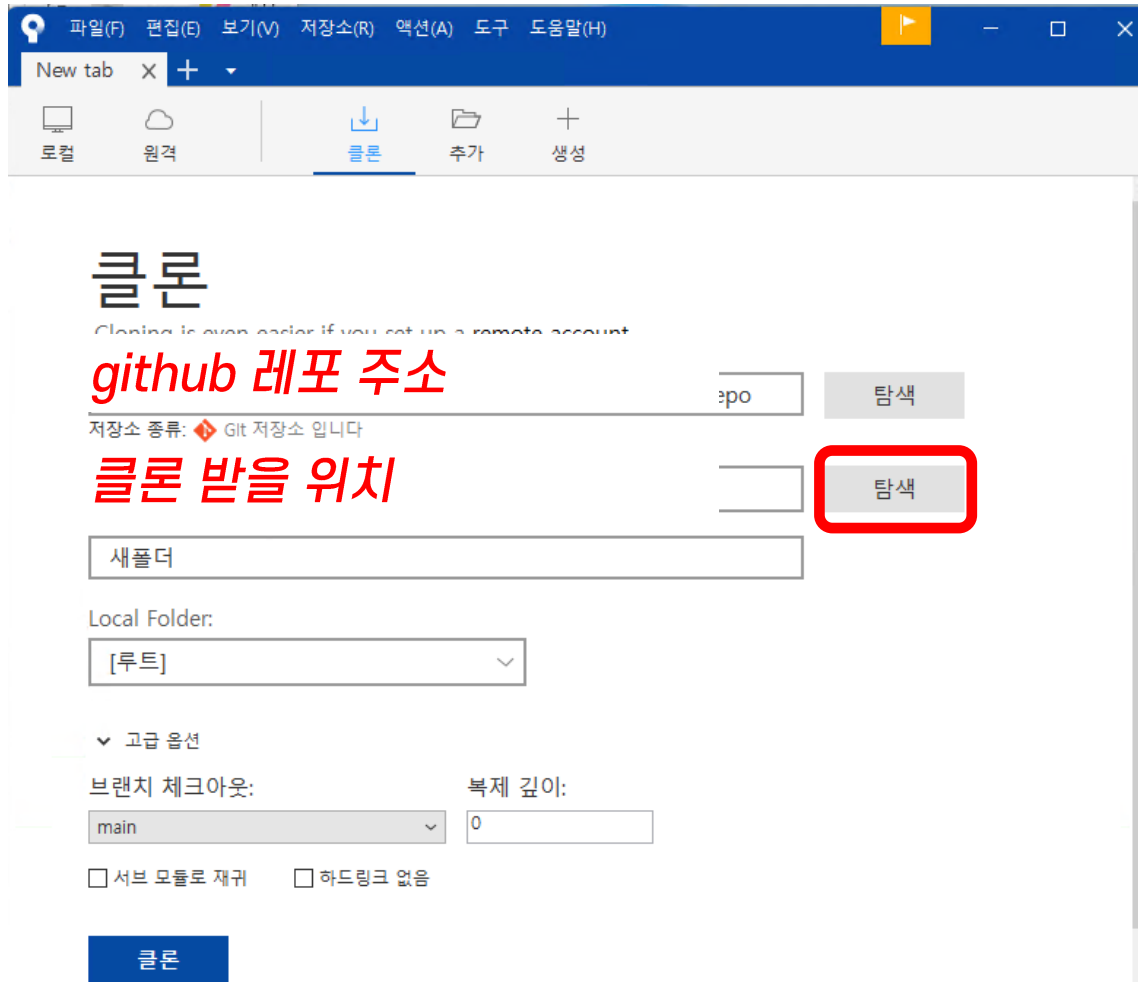


아까 만든 레포 이름 검색

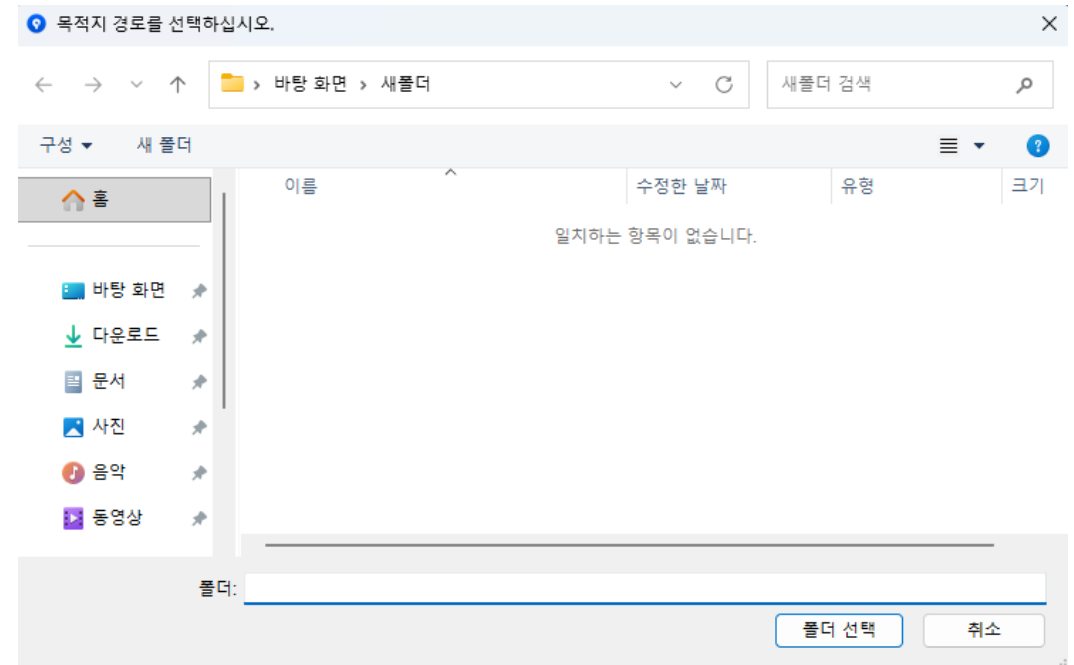
해당 레포의 clone 클릭

SourceTree, 깃프로그램

- 클론: Repo를 내 컴퓨터에 복사해서 갖고옴



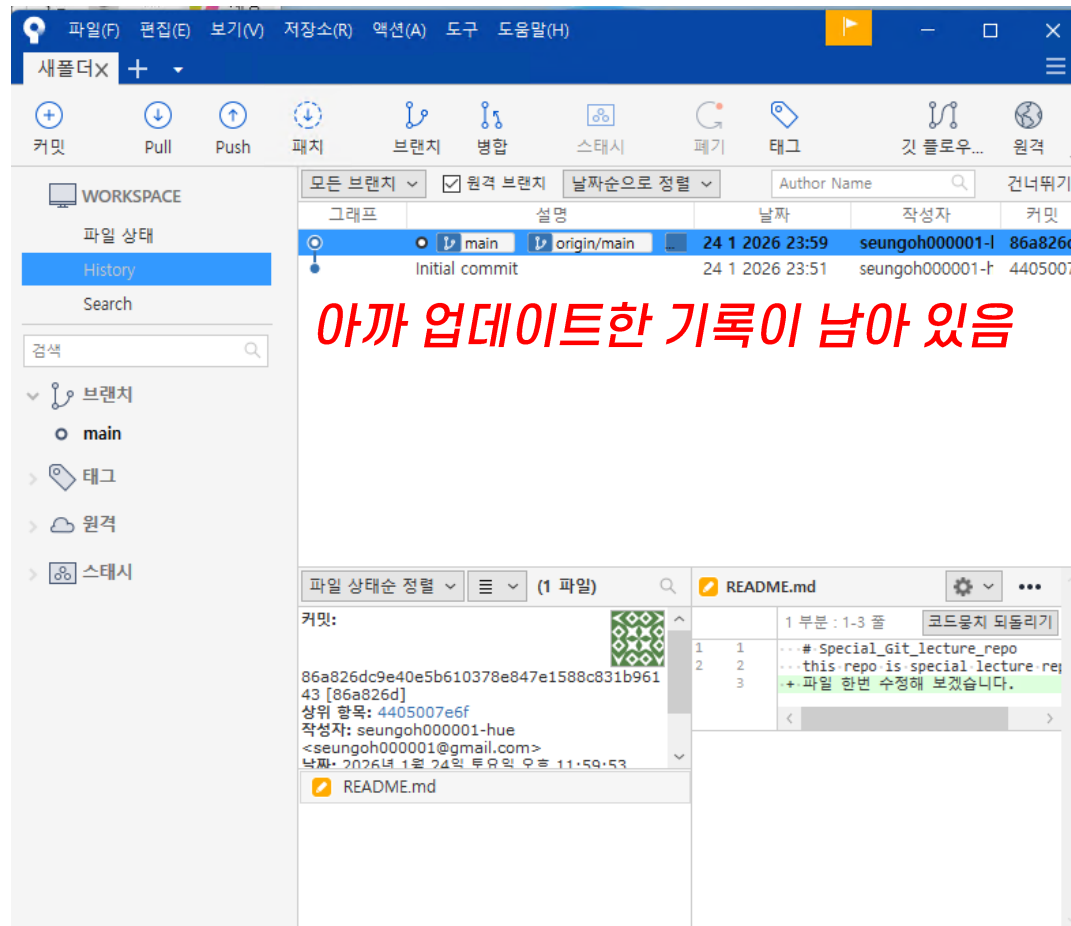
The image shows the 'Clone' dialog box in SourceTree. The title bar says 'New tab X +'. Below the title bar, there are icons for '로컬' (Local), '원격' (Remote), '클론' (Clone), '추가' (Add), and '생성' (Create). The 'Clone' tab is selected. The main area has the title '클론' (Clone) and a subtitle 'Cloning is even easier if you set up a remote account'. There are two input fields: 'github 레포 주소' (GitHub Repo Address) and '클론 받을 위치' (Clone Location). The 'github 레포 주소' field has a '탐색' (Browse) button next to it. The '클론 받을 위치' field has a '탐색' (Browse) button next to it, which is highlighted with a red rectangle. Below these fields is a '새폴더' (New Folder) button. At the bottom, there is a 'Local Folder:' section with a dropdown menu showing '[루트]' (Root). There are also checkboxes for '고급 옵션' (Advanced Options), '브랜치 체크아웃:' (Branch Checkout:), '복제 깊이:' (Clone Depth:), and '서브 모듈로 재귀' (Recursive with submodules) and '하드링크 없음' (No hardlinks).



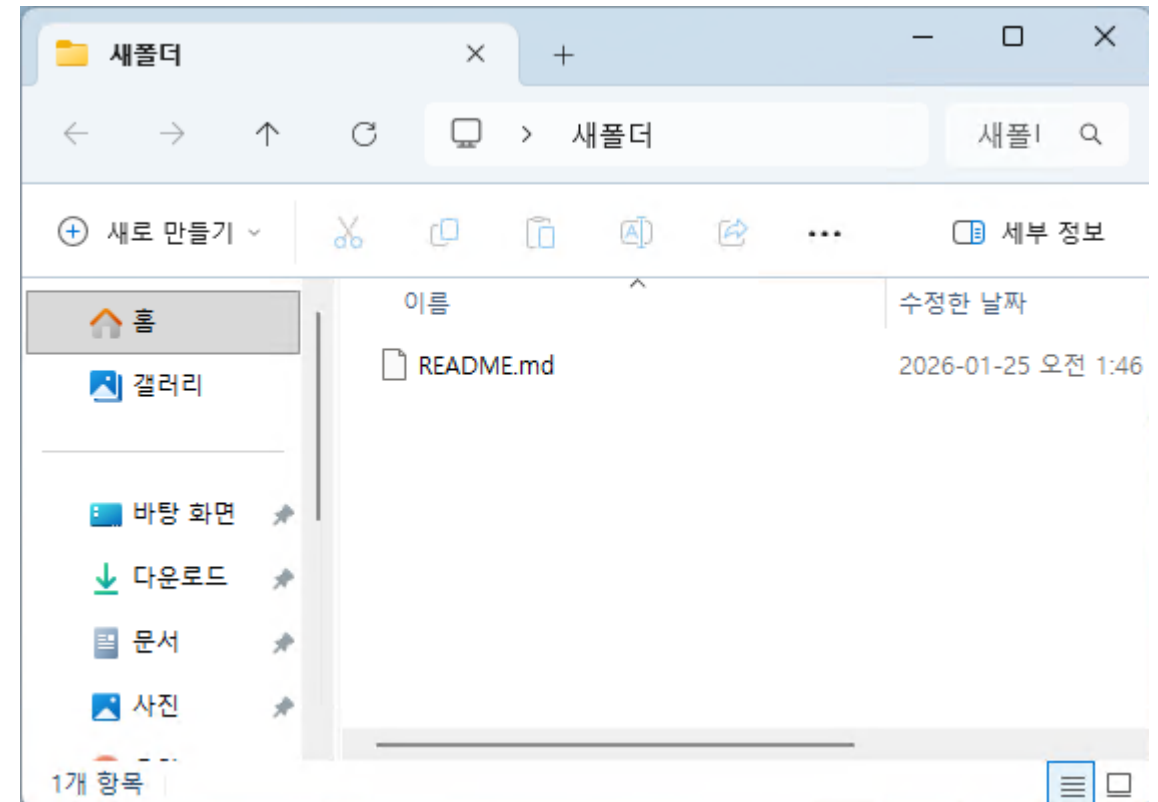
바탕화면에 새폴더 만들어서 폴더 선택

SourceTree, 깃프로그램

- 클론: Repo를 내 컴퓨터에 복사해서 갖고옴



clone된 repo를 볼 수 있음



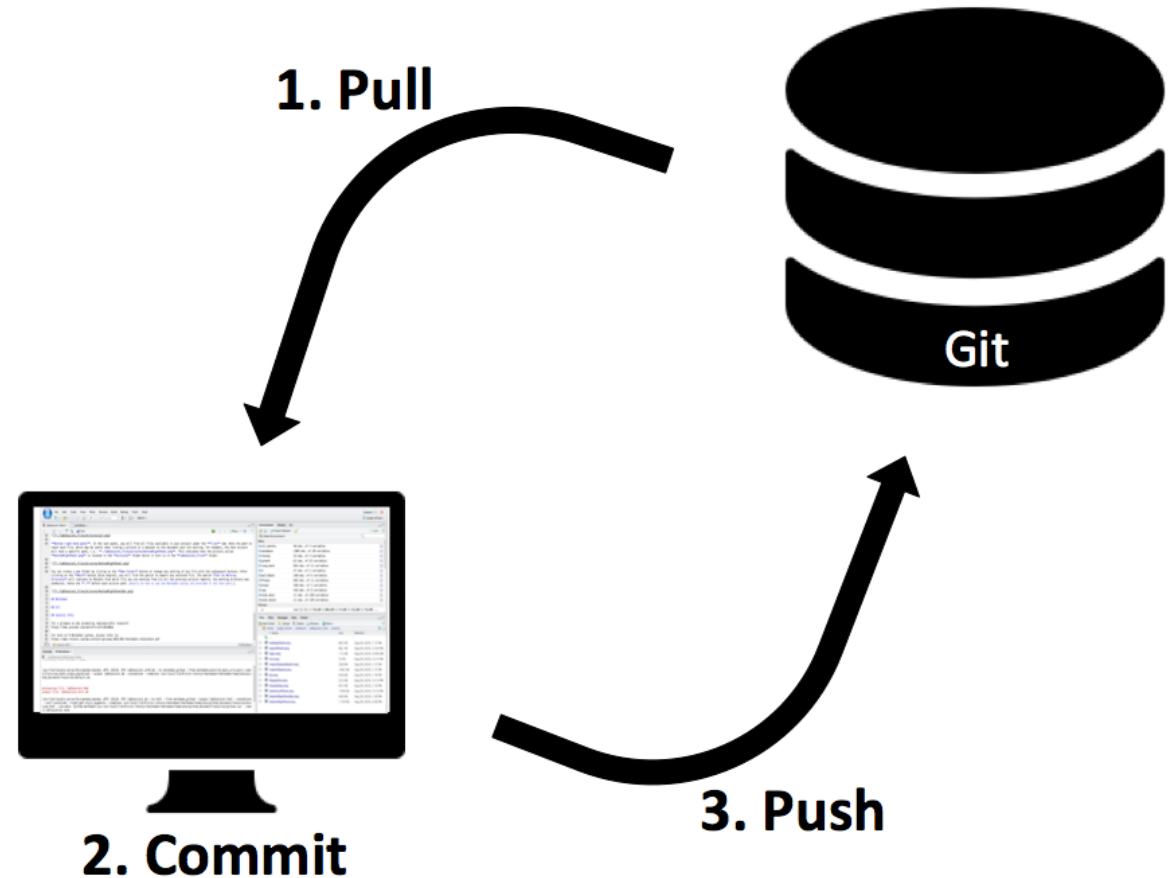
해당 폴더로 가보면 파일도 확인 할 수 있음



Commit & Push

만약 회사에 불이 나면 어찌지?

- Commit이 뭐지?
 - 내 컴퓨터(로컬)에서 변경된 파일을 내 컴퓨터(로컬)에 기록 하는 것
- Push가 뭐지?
 - commit한 파일을 Github(클라우드)에 올리는 것



만약 회사에 불이 나면 어찌지?

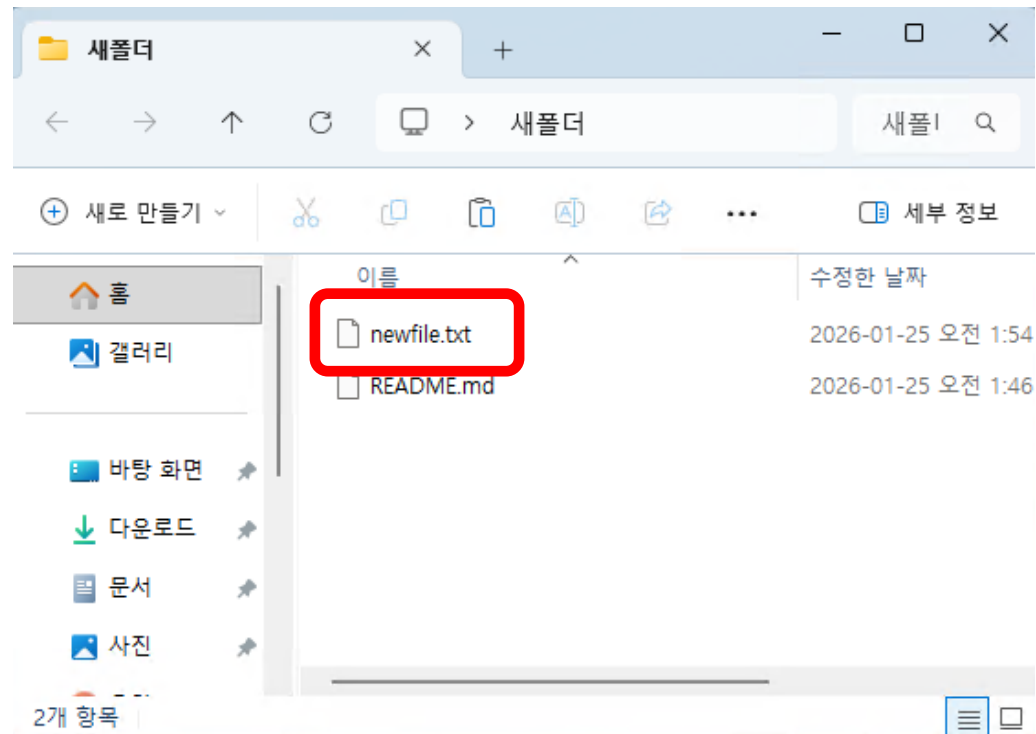
- Commit이 뭐지?
 - 내 컴퓨터(로컬)에서 변경된 파일을
내 컴퓨터(로컬)에 기록 하는 것
- Push가 뭐지?
 - commit한 파일을
Github(클라우드)에 올리는 것



불이 나면, 먼저 커밋 하고, 푸쉬 하고, 건물을 탈출 하세요!

Commit

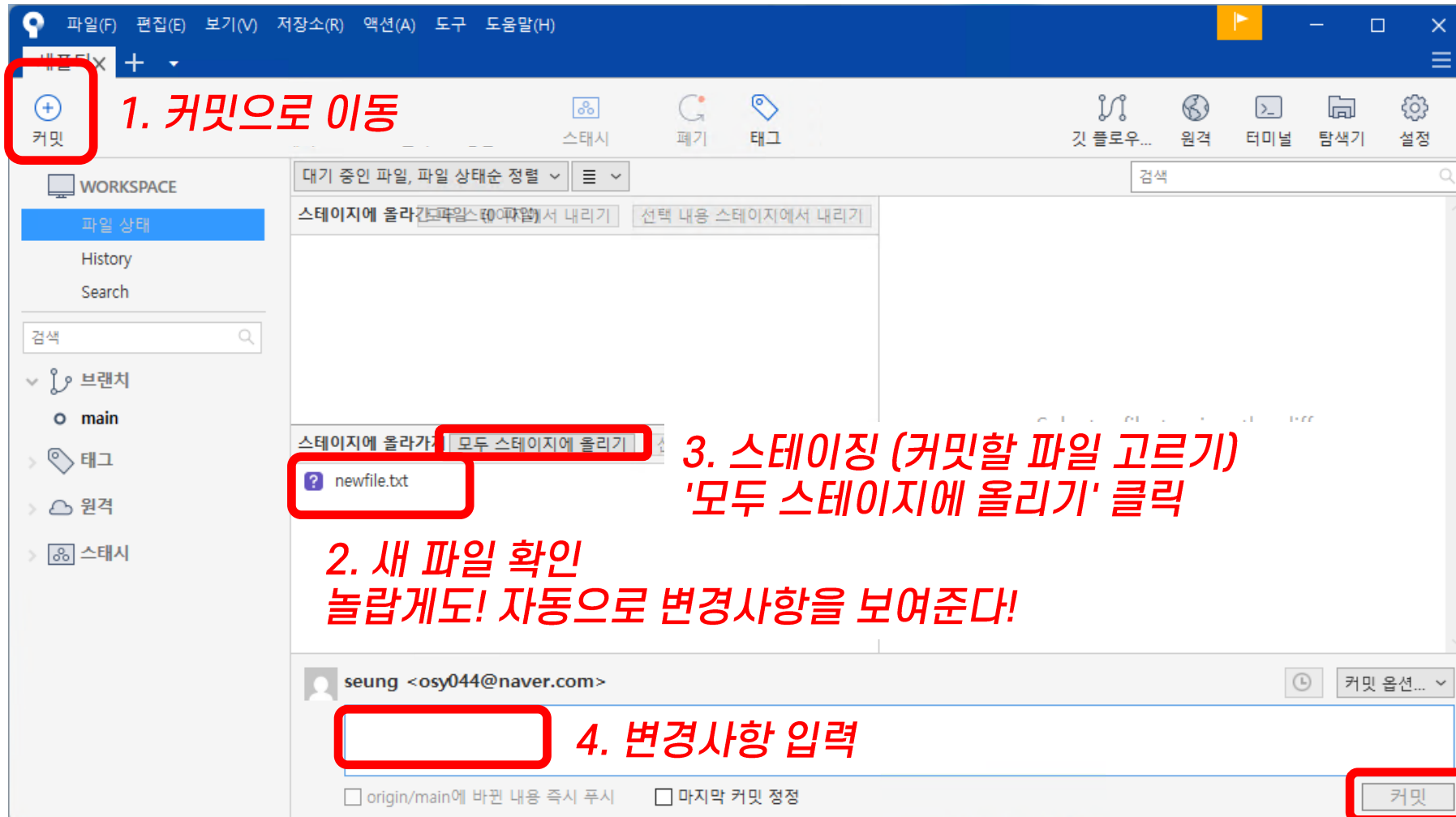
- 이제 폴더에서 새 파일을 추가 해보자!



newfile.txt 추가

Commit

- 추가된 파일 commit 해보기

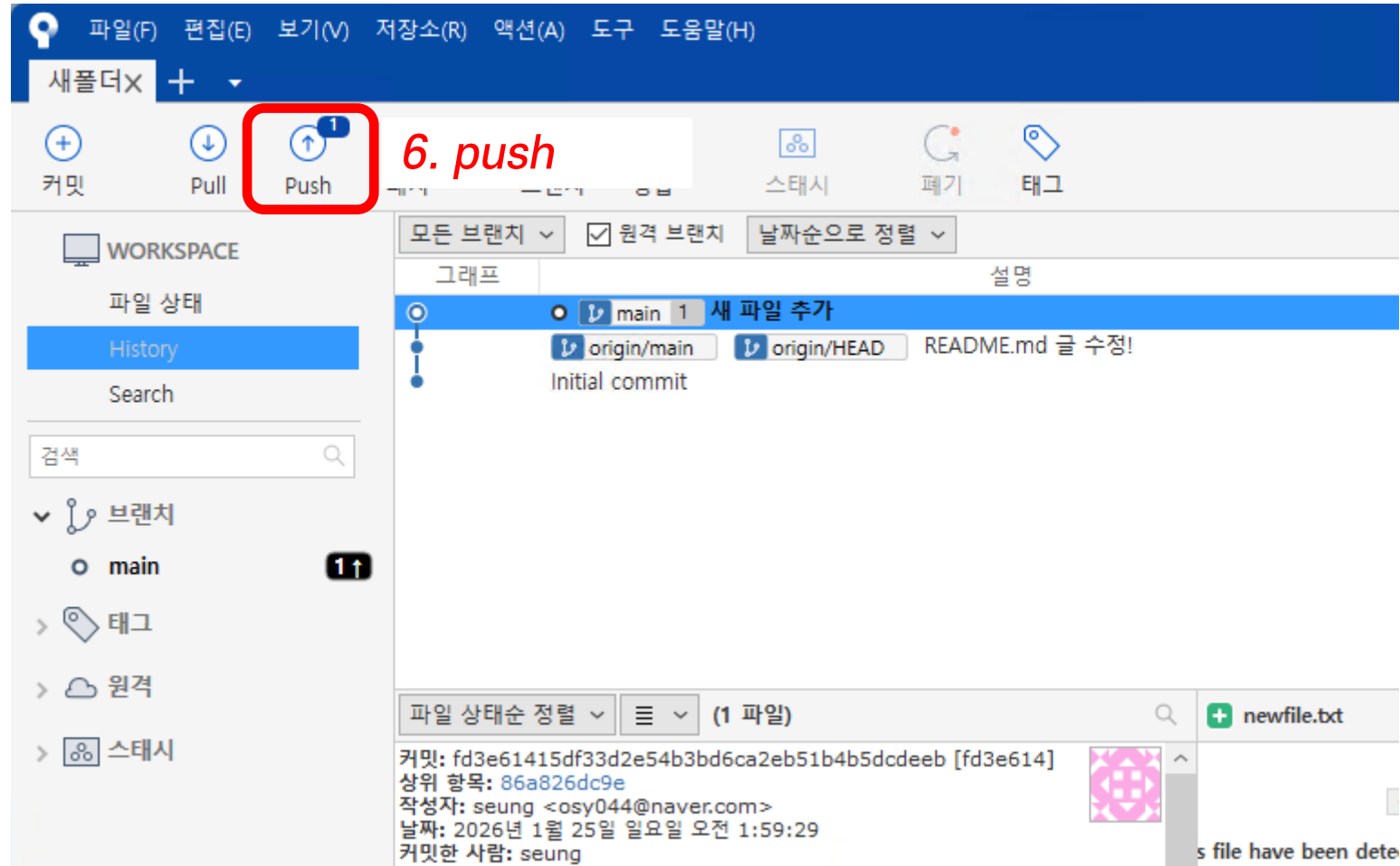


Commit

- Commit을 하더라도 github에 변화가 없음!
- 왜냐하면, 여러분 컴퓨터에 변경사항을 저장한 것이고, 아직 github에는 안올라감

Push

- 추가된 파일 push 해보기



Commit

- Push를 하고나면 변경사항이 github에 반영됨
- 즉, 새로운 파일이 추가됨!

Push

- Push를 하게 되면 최종 파일이 github에 올라가게 됨.
- 그러면 최종본이 바뀌게 됨!
- 즉, 프로젝트가 버전업이 된다!
- push하면 남들이 받아갈 수 있다!
- 되돌리는 방법은 뒤에서 배우지만,
- 이미 받아간 사람까지는 못 막는다



아... 실수...망했다...
(잘못된 코드를 push하면 모두가 힘들어짐)



History

과거 시점으로 돌아가기

History

새폴더X +

커밋 Pull Push 패치 브랜치 병합 스테시 폐기 태그

WORKSPACE

파일 상태

History

History를 누르면 과거 시점을 볼 수 있다.

주의! 더블클릭 = '과거를 구경하는 상태' (HEAD가 main에서 떨어져 나옴) 이 상태에서 파일 수정 · 커밋 금지! 다 봤으면 main 을 더블클릭해서 복귀

모든 브랜치 원격 브랜치 날짜순으로 정렬

그래프

origin/main origin/HEAD

README.md를 수정!

HEAD initial commit

설명

새 파일 추가

25 1 2026 1:59 seung <osy044@r fd3e614

24 1 2026 23:59 seungoh000001-t 86a826d

24 1 2026 23:51 seungoh000001-t 4405007

혹은 다시 최신 시점으로 이동

원하는 시점을 더블 클릭!

다시 폴더를 보면 파일이 원복되어있음!

새로 만들기

홈

갤러리

바탕 화면

다운로드

문서

사진

1개 항목

이름

수정한 날짜

README.md

2026-01-25 오전 2:04

Reset (커밋 되돌리기)

Reset

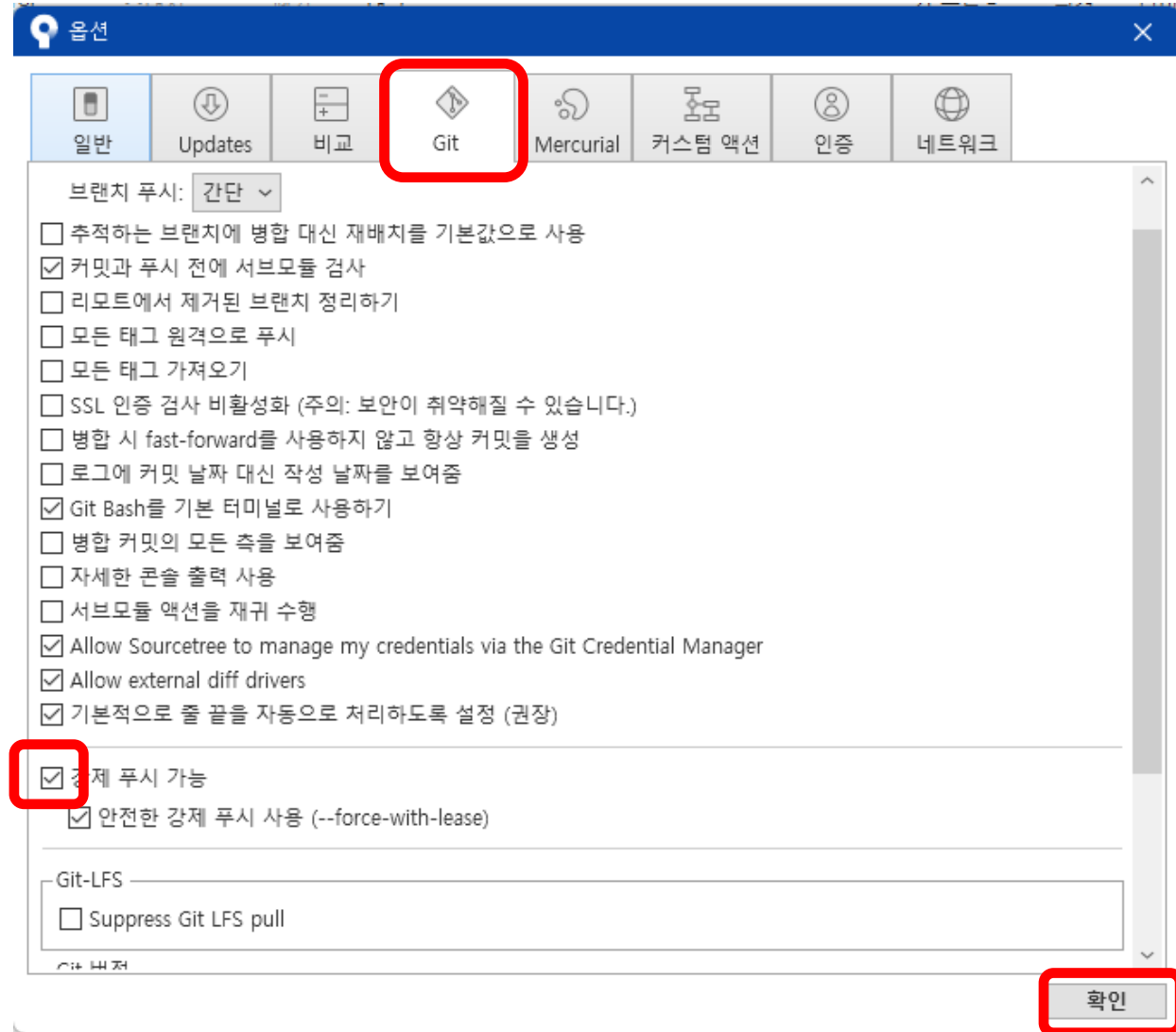
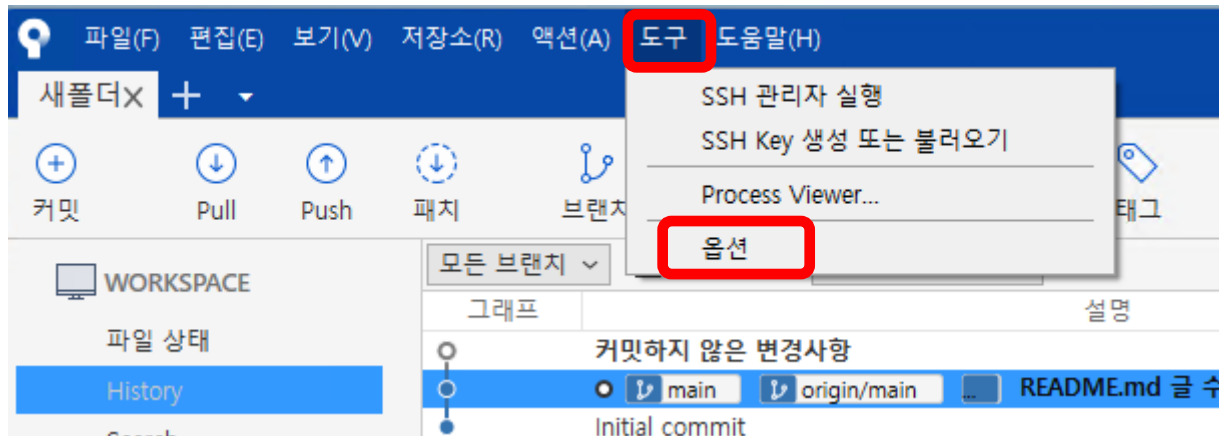
- Reset: 해당 커밋까지 모두 되돌리기
- 실수로 잘못된 파일을 커밋한 경우, 해당 시점까지 되돌리기를 할 수 있다.
- 다만, 해당시점 이후로 commit 했던 파일은 복구가 불가능해진다.



과..과거로 돌아가야해!!

Reset

- Reset = 내 컴퓨터의 커밋 기록을 지우는 것 → 위험!
- GitHub 기록까지 지우려면 '강제 푸시' 필요 (기본은 꺼져 있음)
- 도구 > 옵션 > Git > '강제 푸시 가능' 체크



Reset

The screenshot shows the Git GUI interface. The commit history is displayed in a table with columns for commit hash, date, time, author, and commit message. The commit with hash 86a826d is selected. A context menu is open over this commit, and the option '이 커밋까지 현재 브랜치를 초기화' (Reset to this commit) is highlighted with a red box. A red arrow points to the selected commit in the history table.

1. 되돌리고 싶은 커밋을 마우스 우클릭

이 커밋까지 현재 브랜치를 초기화 클릭

Commit Hash	Date	Time	Author	Commit Message
25 1	2026	16:26	seung <osy044@	481f687
25 1	2026	16:21	seung <osy044@r	4bc864c
24 1	2026	23:59	seungoh000001-f	86a826d
24 1	2026	23:51	seungoh000001-f	4405007

```
1 ...# Special_Git_lecture_repo
2 ...this repo is special lecture repo
3 ...파일 한번 수정해 보겠습니다.
```

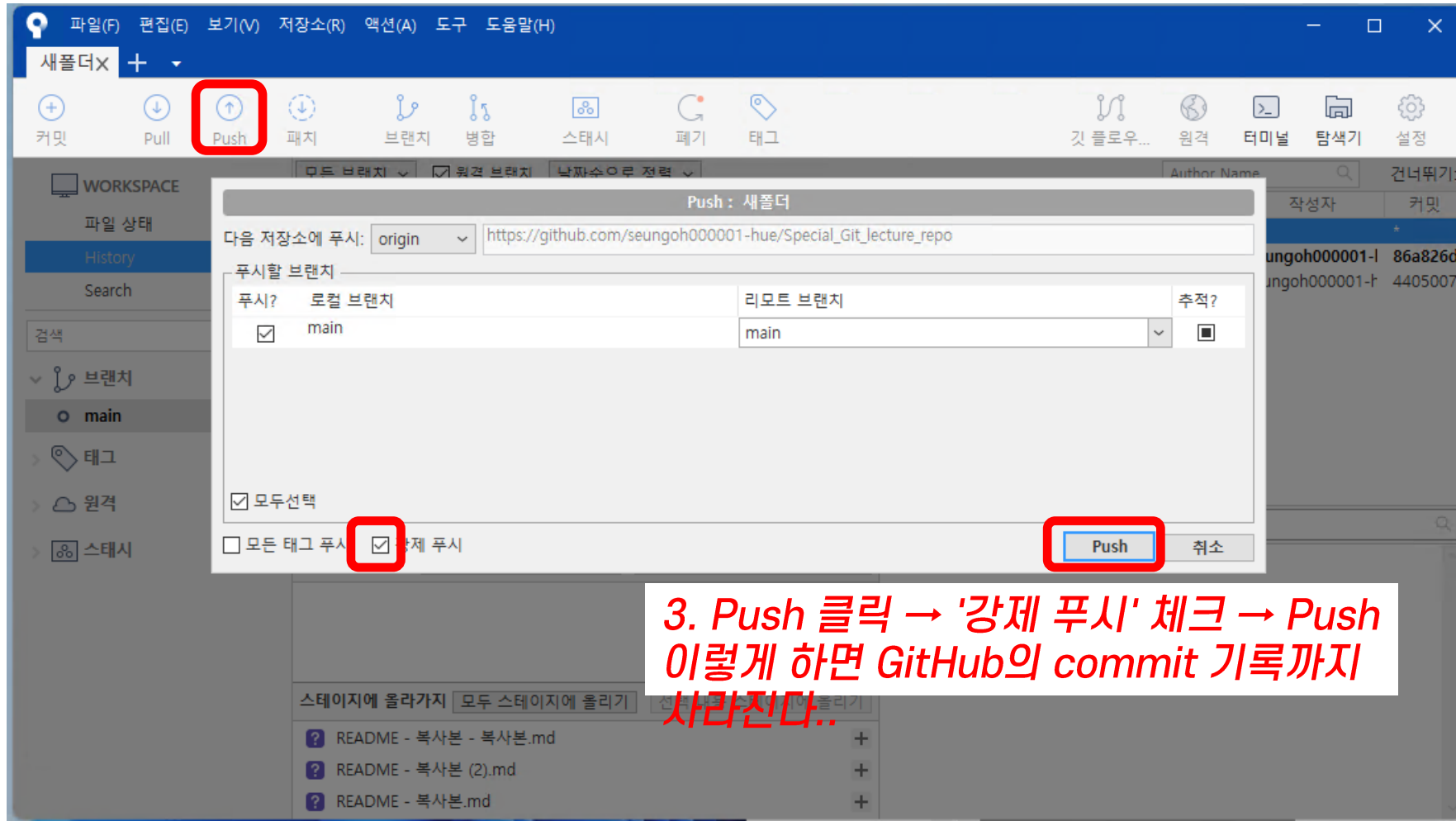
Reset

2. 되돌리기 방식 선택 → Hard 선택 → 확인

- Hard: 커밋 기록 + 파일 내용까지 전부 그 시점으로 되돌림 (우리는 이걸 씀)
- Mixed / Soft: 커밋 기록만 되돌리고, 바뀐 파일 내용은 폴더에 남겨둠
- 초보자는 Hard 하나만 기억하면 됨!



Reset



Revert (특정 커밋 하나만 되돌리기)

Revert

- Revert: 되돌리기
- 실수로 잘못된 파일을 커밋한 경우, 해당 커밋만 다시 되돌리기를 할 수 있다.
- 정확히 말하면, 해당 변경사항만, 다시 원복시키고, 다시 커밋하는것!
- Reset 보다 안전하다.
- 과거 기록을 그대로 남기고 최신 파일에 덮어쓰는것이기 때문
- (하지만 reset이 편하긴 하다..)

Revert

3. push 클릭 → 해당 commit의 수정사항이 다시 덮어쓰기 됨

1. 되돌리고 싶은 커밋을 마우스 우클릭
주의** 첫번째 커밋(initial commit)은 안됨!

2. 커밋 되돌리기 클릭

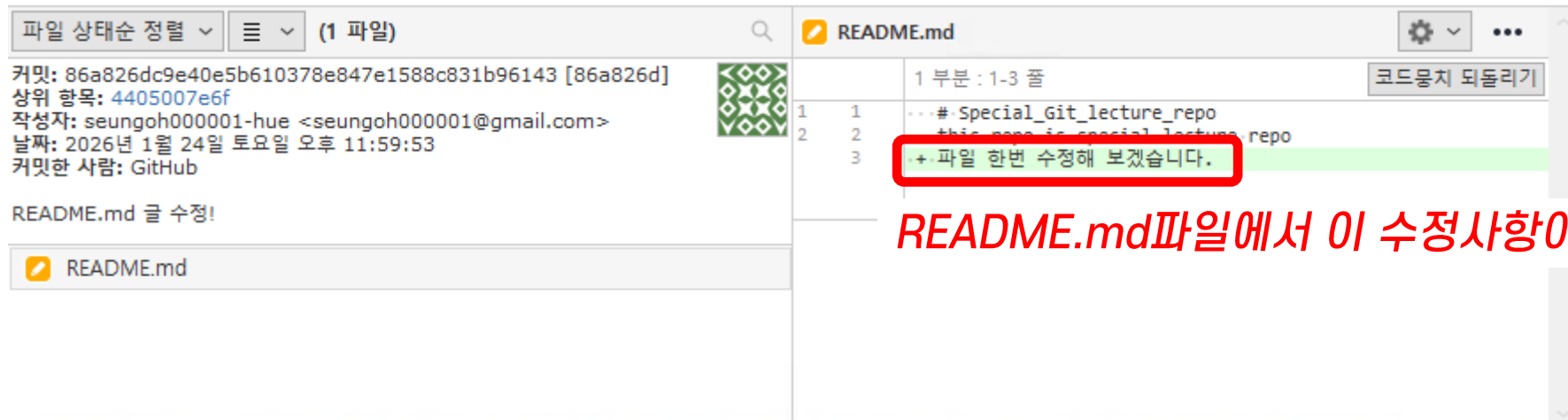
커밋: 86a826dc9e40e5b610378e847e1588c8
상위 항목: 4405007e6f
작성자: seungoh000001-hue <seungoh000001@seungoh000001.com>
날짜: 2026년 1월 24일 토요일 오후 11:59:53
커밋한 사람: GitHub

README.md 글 수정!

README.md

Revert

- 해당 커밋의 수정사항만 원복되었다!

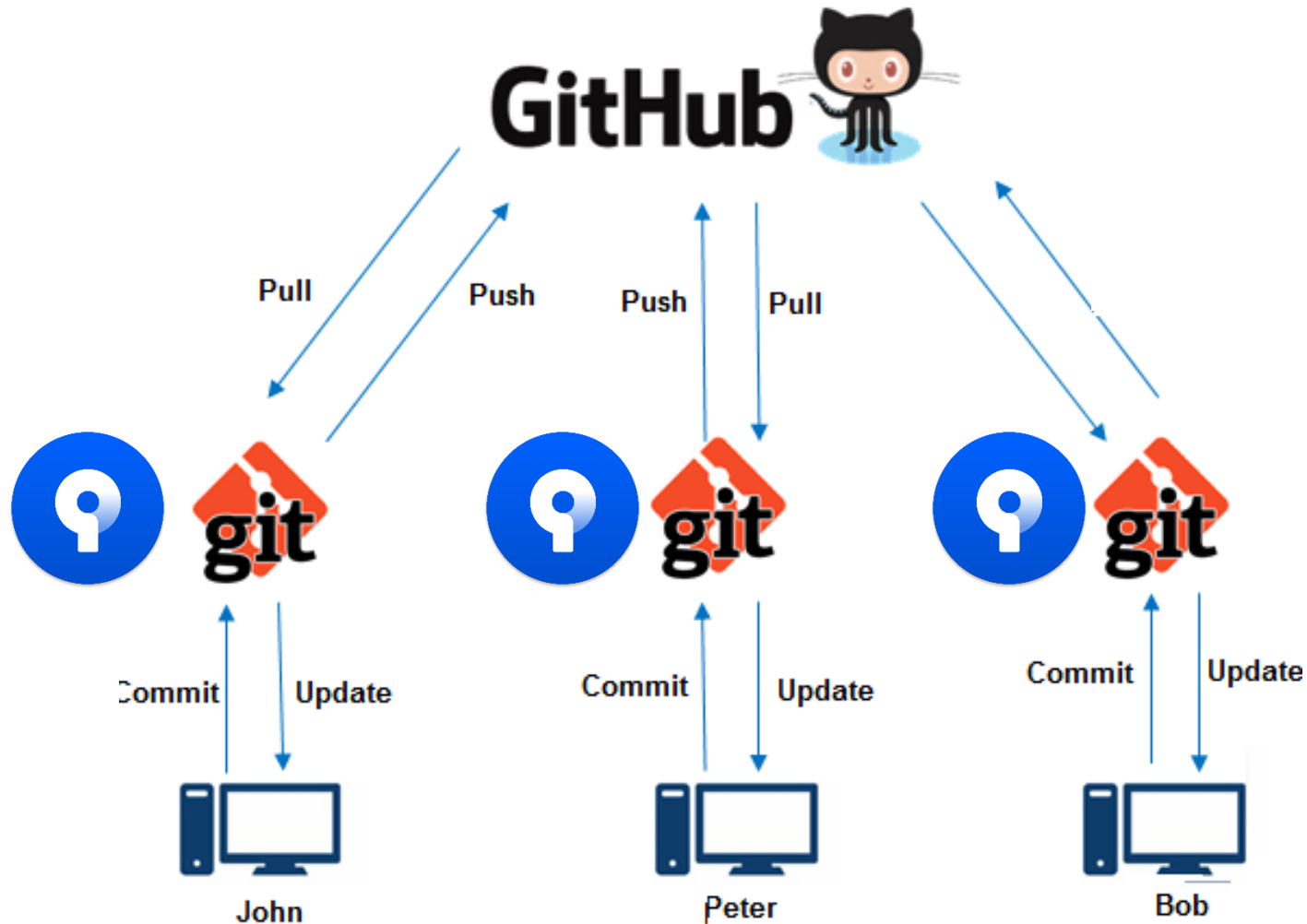




지금까지 한 것 정리

혼자 쓸때는 이정도만 알면 됨

Distributed Version Control System



- Commit
 - 내 컴퓨터에서 변경된 파일을 업데이트(기록) 하는 것
- Push
 - commit한 파일을 Github에 올리는 것
- Pull
 - GitHub의 변경사항을 내 컴퓨터로 받는 것 (Push의 반대)
- Revert
 - 해당 commit 다시 되돌리기

미션1

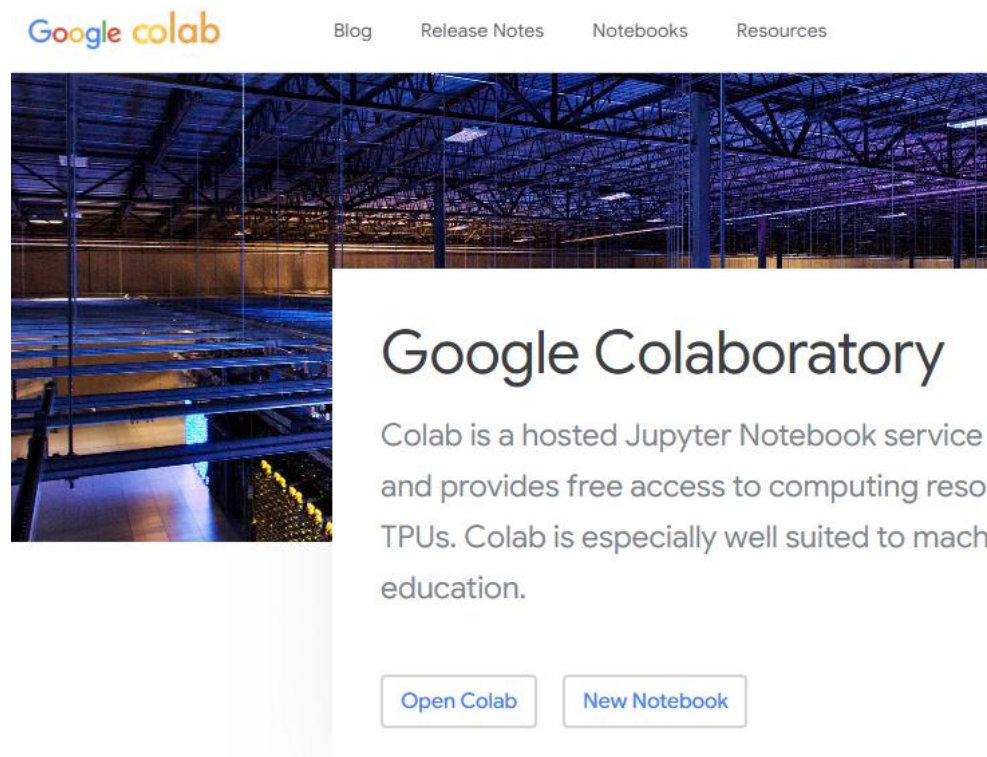
**README.md 파일에 자기 소개
작성해서 commit, push 해보기**

미션2

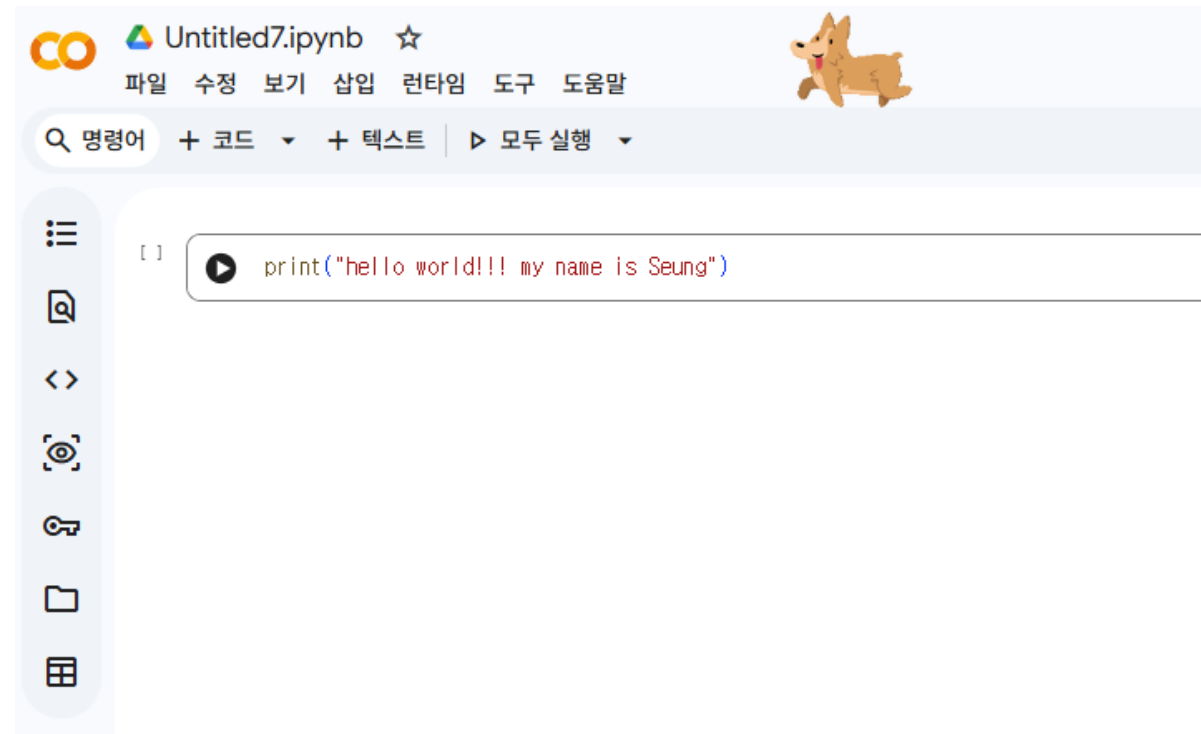
**코랩에서 python 코드 작성해서
내 git repo에 올리기**

Colab

- 구글에서 colab 검색 - new notebook 클릭



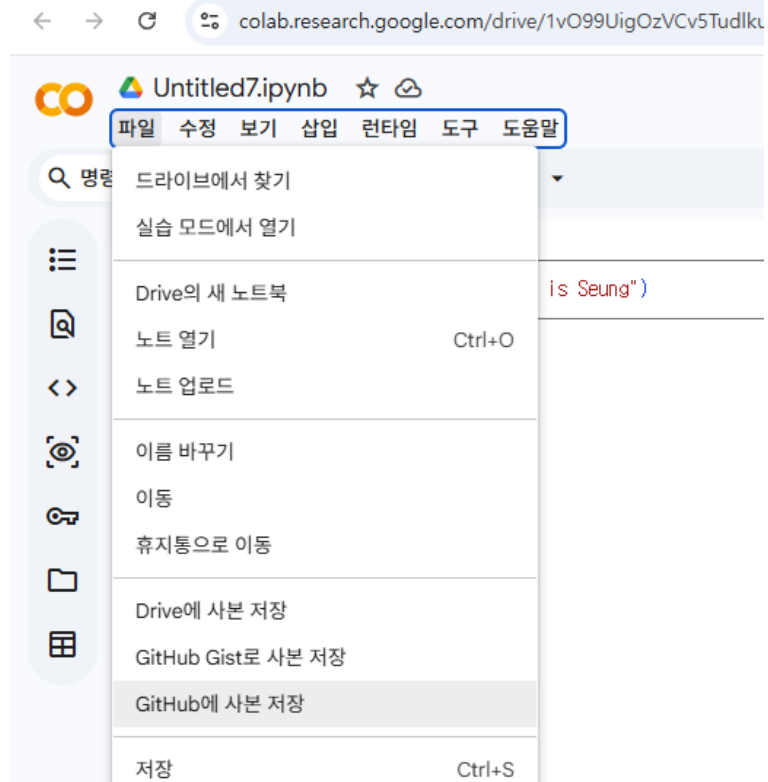
colab 접속



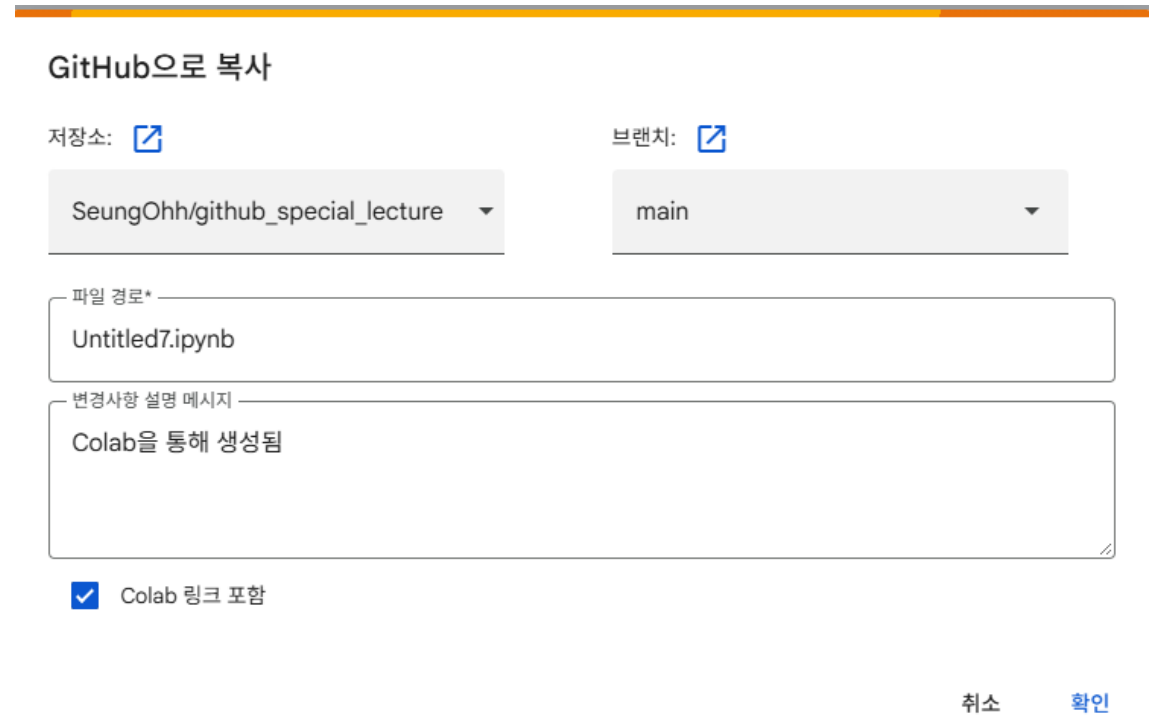
코드 작성

Colab

- 구글에서 colab 검색 - new notebook 클릭



github에 사본 저장



사본 저장

확인!

- 코랩 파일 올라간 것 확인!

 **github_special_lecture** PublicUnpinWatch 0

 main ▾

 1 Branch

 0 Tags

 t

 +



 **<> Code** ▾

 **SeungOhh** Colab을 통해 생성됨

d7264ef · 10 minutes ago

 **2 Commits**

 README.md	Initial commit	2 hours ago
 Untitled6.ipynb	Colab을 통해 생성됨	10 minutes ago

Pull (GitHub의 변경을 내 컴퓨터로 받기)

Pull

미션2를 끝내고 SourceTree를 보면?

- Colab이 GitHub에 직접 올렸기 때문에, 내 컴퓨터(로컬)에는 그 파일이 없다!
- 이걸 클론해서 사용할 수 있음!

Pull

Pull: GitHub의 변경사항을 내 컴퓨터로 받아오기

- 1. Pull 클릭 → 확인
- 2. 폴더를 보면 Colab 파일(.ipynb)이 생겨 있음!
- Push의 반대 방향. 다른 곳(코랩, 다른 컴퓨터, 웹)에서 바꾼 게 있으면 작업 전에 먼저 Pull!
- 안그러면 오류남

